SDN302 – PRACTICE

Customer – Product – Order REST API

GENERAL INSTRUCTIONS

- Use Visual Studio Code, MongoDB Compass, and Postman or Thunder Client.
- Create an Express project using Express Generator. Use customers.json, products.json, and orders.json as the initial data source.
- Create a MongoDB database named sdn302-practice-studentcode-db (replace studentcode with your StudentCode).
- Store the MongoDB connection string, port, and JWT secret in .env. JWT_SECRET must equal your StudentCode exactly.
- Do not commit .env to source control. Do not store JWT in cookies or session storage.
- All required functionality must be implemented as REST API endpoints. No HTML pages, frontend, template engine, or browser-based UI is required.
- Test all endpoints using Postman or Thunder Client. Include appropriate HTTP status codes and JSON responses.

SCENARIO

Build a REST API for an online-store management system. The database has Customer, Product, and Order collections. Implement JWT authentication and REST endpoints for Product and Order management. Customer data is supplied as reference data.

DATABASE SCHEMA

Use Mongoose models and schemas. Enable timestamps where appropriate.

1. Customer

- name: String, required
- phone: String, required
- email: String, required, unique
- address: String, required

2. Product

- name: String, required
- price: Number, required, minimum 0
- category: String, required
- stock: Number, required, minimum 0

3. Order

- customer_id: ObjectId referencing Customer, required
- items: array of { product_id: ObjectId referencing Product, quantity: Number >= 1, price_at_order: Number, total: Number }

- total_amount: Number, required, minimum 0
- payment_method: String, required
- status: String, required
- order_date: Date, required

One Customer may have many Orders. Each Order belongs to one Customer and contains one or more Product references. Preserve or correctly remap ObjectId references when importing the supplied JSON files.

TASK 01 – PROJECT SETUP AND DATA IMPORT

Create an Express project using Express Generator. Connect to MongoDB using Mongoose. Create the database sdn302-practice-studentcode-db and import customers.json, products.json, and orders.json into the corresponding collections. Verify that Order references to Customer and Product are valid.

TASK 02 – JWT AUTHENTICATION

- Create POST /auth/tokens to issue a JWT.
- Validate credentials against members.json: name = storeadmin and pass = Store@123. If the instructor supplies a different members.json, use the supplied data.
- Store the JWT secret in .env as JWT_SECRET, with its value equal to your StudentCode.
- Return HTTP 401 with a JSON error message if credentials are invalid.
- Return the generated JWT in a JSON response if credentials are valid.
- Do not store JWT in cookies or session storage.

TASK 03 – PROTECTED PRODUCT REST API

All Product endpoints below must require a valid JWT sent in the Authorization header as Bearer <token>.

Method	Endpoint	Requirement
GET	/api/products	Return all products.
GET	/api/products/:id	Return one product; return 404 if not found.
POST	/api/products	Create a product after validating required fields.
PUT	/api/products/:id	Update a product; return 404 if not found.
DELETE	/api/products/:id	Delete only if no Order item references the product.

- Implement JWT middleware using jwt.verify(token, process.env.JWT_SECRET).
- Return HTTP 401 if the token is missing, malformed, invalid, or expired.
- Call next() only after successful verification.
- Validate input and return suitable JSON responses and HTTP status codes.
- If an Order references the Product, reject deletion with HTTP 409 and do not delete it.

TASK 04 – ORDER REST API

Implement REST API endpoints for managing Order documents. All endpoints must be protected by the same JWT middleware.

Method	Endpoint	Requirement
GET	/api/orders	Return all orders, including customer and product details where appropriate.
GET	/api/orders/:id	Return one order by ID; return 404 if not found.
POST	/api/orders	Create an order using a valid customer and one or more valid products.
PUT	/api/orders/:id	Update an existing order; return 404 if not found.
DELETE	/api/orders/:id	Delete an order by ID; return 404 if not found.

- Validate that customer_id refers to an existing Customer and each product_id refers to an existing Product.
- Each item quantity must be an integer greater than or equal to 1.
- Set price_at_order from the current Product price on the server. Calculate each item total and total_amount on the server; do not trust totals sent by the client.
- Reject an order if requested quantity exceeds available stock.
- Require payment_method, status, and order_date (or assign order_date on the server if the API design specifies this).
- Use Mongoose populate or equivalent queries to return useful customer/product details in GET responses.
- Return appropriate JSON messages and status codes for validation errors, missing records, and successful CRUD operations.

POSTMAN / THUNDER CLIENT TEST CHECKLIST

- POST /auth/tokens: valid credentials return a JWT; invalid credentials return 401.
- Protected endpoints return 401 when the Authorization header is missing or the token is invalid.
- All Product CRUD operations work with a valid Bearer token.
- Deleting a Product referenced by an Order returns 409 and does not delete the Product.
- All Order CRUD operations work with valid input and a valid token.
- Invalid Customer/Product IDs, missing fields, invalid quantities, and missing records return appropriate errors.
- Order totals are calculated by the server, not accepted from the client.

SUBMISSION CHECKLIST

- Project starts without syntax or runtime errors.
- MongoDB contains the three supplied collections and their references are valid.

- .env contains the MongoDB URI, port, and StudentCode-based JWT_SECRET and is not committed.
- All functionality is available through REST API endpoints; no frontend or view templates are required.
- All endpoints have been tested using Postman or Thunder Client.